FORECASTING HIGHWAY TRAFFIC SPEED 15–60 MINUTES AHEAD

Phase 1 Report · Advanced Machine Learning & Deep Learning (CSAI)

Project 4 · Multi-Agent Traffic Flow Control via Graph Attention Networks and LWR Models

Team T082 · Section B · Kartik Madaan, Arhan Alam, Chirag Lalwani, Siddhanth Sadshiv Raikar, Abhijeet Dey

Newton School of Technology, Rishihood University · October 2026

Summary

We tried to predict **how fast traffic will be moving at each of 207 highway sensors in Los Angeles, 15, 30 and 60 minutes from now**, using the last hour of sensor readings, the clock, and what the neighbouring sensors are doing.

What we found, in five points:

1. **8% of the readings are broken, not slow.** A reading of exactly 0 mph is a sensor that was offline. We marked those cells "unknown" — nothing deleted, nothing guessed — and the models handle them directly.
2. **Traffic follows the clock.** Weekdays have two rush hours; at 17:00, **35%** of sensors are jammed. A sensor's speed is **84%** correlated with its own speed one hour earlier, so "what it usually does at this time of week" and "what it did in the last hour" are both strong clues.
3. **Neighbours matter, and physics says why.** A jam grows backwards, like a queue, at 15–20 km/h (the LWR model). Sensors under 1 km apart have a speed correlation of **0.85**; 10 km apart, **0.19**. We turned this into eight neighbour features.
4. **Our best model works.** XGBoost with neighbour features has an average error of **2.87 / 3.32 / 3.78 mph** at 15 / 30 / 60 minutes, against 5.33 mph for "the speed stays the same" and 4.19 mph for "the usual speed" at 60 minutes. A published deep graph model (DCRNN) reaches 3.60 at 60 minutes, so a classical model gets close.
5. **Neighbour information helps most in rush hour and least at 60 minutes.** Adding the eight neighbour features lowers the error by 4.1 / 3.3 / 1.8%, three times more in the peaks than off-peak. One hop of neighbours only "sees" 5–15 minutes ahead; that fading is exactly the gap a multi-hop graph network should close in Phase 2.

---

1. Why we chose this problem

- **It matters in real life.** A traffic-signal controller or ramp meter has to act before the jam arrives. Knowing the speed now is not enough; it needs the next hour. Phase 1 builds that forecast; Phase 2 will add the control part, in simulation only.

- **The data is real and public.** We used METR-LA (Li et al., 2018): 207 loop detectors on Los Angeles County highways, a reading every five minutes from 1 March to 27 June 2012, plus a road map that says which sensors are neighbours. It is the standard benchmark, so our numbers can be compared with published ones.
- **It is a real machine-learning challenge.** The data is a time series, so a random train/test split would cheat. Eight percent of readings are broken. Errors concentrate in rush hour, where the forecast matters most. And the project brief asks a specific question: how much does a sensor gain from looking at its neighbours?

Traffic has been modelled as a fluid since the 1950s: Lighthill and Whitham (1955) and Richards (1956) showed that cars are conserved along a road and that disturbances travel as waves — forwards when the road is empty, **backwards** when it is congested. Short-term forecasting has a long history of statistical models (Vlahogianni et al., 2014), with two simple predictors — "the usual speed" and "the speed stays the same" — remaining the yardsticks every method is judged against. Tree ensembles such as gradient boosting beat linear models for travel-time prediction (Zhang & Haghani, 2015). The decisive step in recent years was to treat the sensors as a graph: DCRNN (Li et al., 2018) released the METR-LA benchmark we use, and graph attention models (Veličković et al., 2018; Zhang et al., 2019) let the model learn how much to listen to each neighbour. On testing, Bergmeir and Benítez (2012) show that ordinary cross-validation is invalid for time series and that one must split by time. Phase 1 uses only classical models; its job is to build the baseline and the honest test bench that the Phase 2 graph model will be compared against.

---

2. The data

What the dataset looks like

METR-LA is **one big table**: **34,272 rows** (one per five-minute slot over 119 days) and **207 columns** (one per sensor). Each cell is the average speed in miles per hour. There are no missing rows: every five-minute slot from 1 March to 27 June 2012 is present.

It comes with a **road map**: where each sensor is, the **driving distance** between sensor pairs, and from those an adjacency matrix that says how strongly each sensor is linked to each neighbour (nearer = stronger; links longer than a cut-off are dropped). We rebuilt this matrix from the raw distance table and checked that it matches the published one exactly. The graph has **1,515 directed links**, about **7 neighbours per sensor**, and one sensor with no neighbours.

207	34,272	7.1 M	1,515
sensors (columns)	five-minute slots (rows)	readings in total	road links between sensors

What one example looks like. For one sensor at one moment, we describe the situation with **34 input numbers** (Section 5) and the answers are the speed at that sensor **15, 30 and 60 minutes later**.

Data quality: the readings of 0 mph

8.1% of all cells contain exactly 0 mph. A highway never averages exactly zero over five minutes, so these are not jams — they are sensors that were switched off. Three things confirm it:

- The zeros sit between normal readings: a sensor reads 69, then 0, 0, 0, then 67 within 25 minutes.
- They come in runs: **25,963 runs**, median 2 slots, but the longest **1,683 slots** (almost six days).
- On **6.3%** of time slots **every** sensor reads 0 at once — a data-collection failure, not 207 simultaneous jams.

So we **relabel every 0 as "unknown"**. Nothing is deleted: the table keeps all its rows and columns, and the worst sensor (20% unknown) is kept. Nothing is invented either: we do not fill the gaps with guessed values, because the long runs would carry stale readings through rush hours, and guessed *answers* would corrupt the scoring. The tree models accept unknown inputs directly; an example whose answer is unknown is simply skipped, both in training and in scoring (the same convention as the DCRNN paper).

After this cleaning the speeds have a mean of **58.5 mph** and a median of **63.2 mph**; **9.0%** are below 35 mph, the threshold Caltrans uses for "congested". The distribution has two humps — free flow around 65 mph and a congested mode around 20–30 mph — which is the two-regime picture from traffic physics showing up in the data.

Splits

We cut the four months **by date**, never by shuffling, with a **one-hour gap** at each boundary so that no training answer lies inside the test period:

GROUP	SLOTS	DATES	USED FOR
Train	23,967	1 March – 23 May	Teaching the models
Validation	3,403	23 May – 4 June	Deciding when to stop training (nothing else)
Test	6,831	4 – 27 June	Final check, used once

How we got the number of examples

STEP	TRAIN	VALIDATION	TEST
1. Five-minute time slots	23,967	3,403	6,831
2. Readings (slots × 207 sensors)	4,961,169	704,421	1,414,017
3. Exactly 0 mph → marked unknown	-353,732 (7.1%)	-49,317 (7.0%)	-171,637 (12.1%)
4. Inputs per example: 34 numbers (26 about the sensor itself + 8 about its neighbours)			
Examples whose answer (speed 60 min later) is known	**4,607,444**	**655,115**	**1,242,308**
…of which jammed (below 35 mph)	8.6%	8.7%	10.3%

Two things to notice:

- The test weeks have **more unknown readings** (12%) than the training weeks (7%) — several whole-network outages fall in June — which makes "masked" scoring (Section 6) essential.
- The test weeks also have **more jams** (10.3% vs 8.6%), partly because mid-June includes the start of the summer holidays. The test period is slightly harder than the training period, not easier.

---

3. How we avoided cheating (data leakage)

"Leakage" means the model sees information it would not have in real life, which makes its score look better than it really is. We took six steps against it:

1. **Split by date, not at random.** Two readings five minutes apart are nearly identical. A random split would put near-copies of the test data into training, and the score would look far better than it really is (Bergmeir & Benítez, 2012).
2. **Leave a gap.** A one-hour gap at each boundary, so no training example's answer falls inside the validation or test period.
3. **Only look backwards.** Every input is built from the past: the sensor's last hour, and the neighbours' speeds now, 5 and 10 minutes ago. The answer is only ever used as the answer.
4. **Compute "the usual speed" from training weeks only.** The time-of-week average for each sensor is a strong feature; it is calculated on March–May and then simply looked up for June.
5. **Fit the scaler on train only.** The linear model needs standardised inputs; the scaling constants come from training rows.
6. **Use the test set once.** All model settings were fixed in advance; the validation weeks only decide when to stop training; the test weeks were scored once, at the end.

Two of our eight automated tests check this directly: that no test sample's input window reaches into the training targets, and that the "usual speed" profile does not change when the test data is altered.

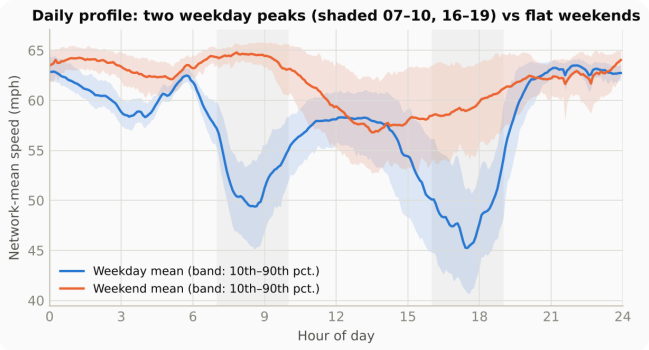
---

4. What we learned from exploring the data (EDA)

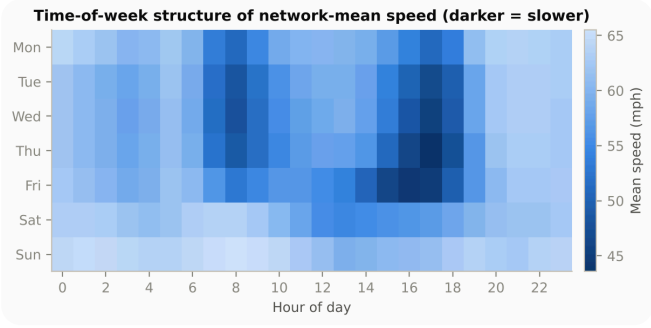
Off-peak vs rush hour

MEASUREMENT	OFF-PEAK	WEEKDAY RUSH HOUR	WEEKEND
Mean speed (mph)	59.6	50.1	61.7
Spread of speeds, standard deviation (mph)	10.6	18.5	10.6
Readings below 35 mph ("jammed")	5.2%	26.5%	5.0%
Sensors jammed at 17:00 on a weekday	**—**	**35%**	**—**

Rush hour = weekdays 07–10 and 16–19. The average speed drops by about 10 mph, the spread almost doubles, and the share of jammed readings goes from 5% to 26%. Weekends look like off-peak all day. The evening peak is deeper than the morning one.



Average speed by time of day: two weekday rush hours (shaded), flat weekends.



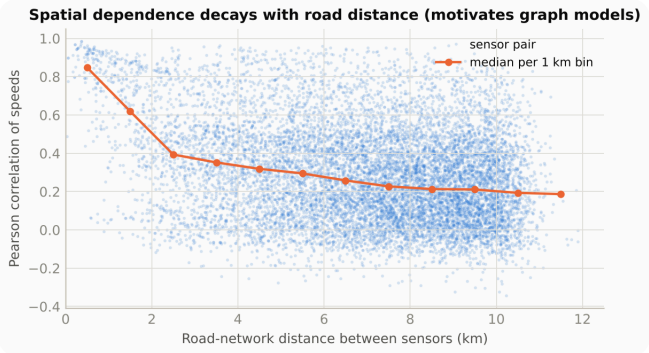
Speed by day of week and hour: the same pattern every week.

How speed relates to itself and to the neighbours

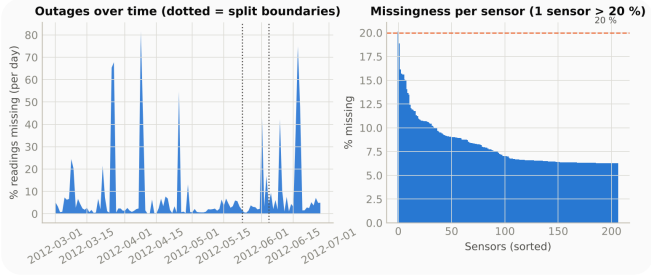
"r" (correlation) is a number from -1 to +1 that shows how much two things move together; 0 means no link.

PAIR	R	WHAT IT MEANS
A sensor's speed & its own speed one hour earlier	0.84	The last hour is a strong input
A sensor's speed & its own speed 24 hours earlier	0.64	A daily cycle: "the usual speed" is a strong input
Two sensors under 1 km apart (by road)	0.85	Close neighbours move together
Two sensors 5–6 km apart	0.29	The link fades with distance…
Two sensors 10–11 km apart	0.19	…so only the near neighbourhood is informative

These match real-world physics (Section 5), which tells us the data is trustworthy. Each sensor also has its own personality: congestion returns every weekday, but its depth and timing differ from sensor to sensor. That is why we train **one model per sensor**.



20,000 sensor pairs: correlation of their speeds falls with the road distance between them (orange = median per km).



Share of unknown readings per day (left) and per sensor (right); spikes are whole-network outages.

5. Feature engineering

A model learns from examples of the form "situation → what happened next". For every sensor at every five-minute slot we describe the situation with **34 numbers**, in five groups:

GROUP	FEATURE(S)	HOW WE MADE IT	WHY
A · own history	`lag_0` … `lag_11` (12)	The sensor's speed now and in each of the previous 11 slots	The last hour is 84% correlated with the next
B · summary	`win_mean`, `win_std`, `win_min`, `win_max`, `win_frac_missing`, `trend_15min` (6)	Mean, spread, min and max of that hour; share of it unknown; change over the last 15 min	Tells the model whether speed is steady, falling, or unreliable
C · the clock	`tod_sin`, `tod_cos`, `dow_sin`, `dow_cos`, `is_weekend`, `hour` (6)	Time of day and day of week as sine/cosine pairs, plus a weekend flag and the hour	23:55 and 00:05 should look close; Sunday should sit next to Monday
D · usual speed	`ha_now`, `dev_from_ha` (2)	Average speed of this sensor at this time of week (training weeks only); current speed minus that	"Normal for a Tuesday at 5 pm", and how far from normal we are now
E · neighbours	`up_mean` ×3, `down_mean` ×3, `nbr_min`, `nbr_frac_congested` (8)	Weighted average speed of upstream and of downstream neighbours now, 5 and 10 min ago; slowest neighbour; share of neighbours below 35 mph	Jams travel between sensors (below)
—	**speed +15, +30, +60 min**	**The same table, shifted forward in time**	**The answers we want to predict**

The brief mentions flow and occupancy as possible inputs; the public METR-LA release contains speed only, so every feature is built from speed. Through the speed–density relation, speed is a proxy for density, so most of the information about the traffic regime is still there.

Why the neighbour features: jams grow backwards

Think of a queue at a ticket counter: new people join at the back, so the queue grows *backwards*. A traffic jam does the same. In the Lighthill–Whitham–Richards (LWR) model, cars are conserved along the road, and speed falls as the road gets denser (Greenshields, 1935). A small disturbance then travels at the **wave speed** $c(\rho) = v_f(1 - 2\rho/\rho_j)$, where v_f is the free-flow speed and ρ_j the density at which nothing moves. When the road is emptier than half the jam density, c is positive and information moves forward with the cars; when it is more congested, c is **negative** and information moves backwards, against the cars. The tail of a real jam moves upstream at about **15–20 km/h** (Treiber & Kesting, 2013).

Three decisions follow:

- Whether sensor *i* will be congested soon depends on its **downstream** neighbours (a queue growing back towards it) and its **upstream** neighbours (traffic about to arrive). So we keep separate upstream and downstream averages.
- Nearer neighbours matter more, so the averages are weighted by the road-graph link strength, and unknown neighbours are left out of the average.
- A backward wave covers 10–20 km in an hour — more than one hop in our graph. One-hop features should therefore help most at short horizons and run out of reach at 60 minutes. A multi-layer graph network (Phase 2) can look several hops out.

Local vs spatial

FEATURE SET	WHAT IT CONTAINS	NUMBER OF FEATURES
Local	Groups A–D: what the sensor knows about itself	26
Spatial	Local + group E: the neighbours	34

Every learned model is trained **twice**, once with each set. Same rows, same split, same settings — the only difference is the eight neighbour columns. That is what makes the comparison in Section 7 a fair measurement of what the neighbours are worth.

---

6. The models

We trained five models, from the simplest to the most powerful. Each one answers one question.

MODEL	WHAT IT DOES	THE QUESTION IT ANSWERS
Historical average	Predicts the sensor's usual speed at this time of week. No learning.	How far does "the usual" get you?
Persistence	Predicts the speed stays what it is now. No learning.	How far does "right now" get you?
Ridge regression	Gives each input a weight and adds them up (a straight line); a penalty keeps the weights small.	Is a simple linear model enough?
XGBoost	Builds many small decision trees one after another, each fixing the last one's mistakes.	Do combinations of clues help ("5 pm **and** speed dropping **and** the sensor ahead is slow")?
Random Forest	Builds 100 trees on random samples of the data and averages them; one forest predicts all three horizons at once.	Is XGBoost's result just one method's quirk?

One model per sensor. Sensors have different personalities, so we train a separate model for each of the 207 sensors (and, for XGBoost, one per sensor per horizon: **621 models**, about 12 minutes on two CPU cores).

Settings were chosen in advance from standard practice, not tuned on the test set. XGBoost: tree depth 6, learning rate 0.05, at least 5 examples per leaf, 80% of rows and columns per tree, squared-error loss, at most 600 trees with **early stopping** — stop when the validation error has

not improved for 40 trees. On average only **129** of the 600 trees were kept, so the models are not over-fitting. Random Forest: 100 trees, depth at most 14, at least 5 examples per leaf, half the features considered at each split. Ridge: penalty $\alpha = 1$, inputs standardised, unknown inputs filled with the sensor's usual speed (the only model that needs filling).

Both tree libraries accept **unknown inputs**: at every split, the tree learns which way to send a missing value. That is why the masked outages need no filling.

How we score the models

With x the true speed and $\hat{x}$ the prediction, over all test examples whose true speed is known:

- **MAE** (mean absolute error) — the average size of the mistake, in mph. Our headline number: easy to read, robust, and what the literature reports for this dataset.
- **RMSE** (root mean squared error) — punishes big misses more. In traffic, the big miss is a jam that was not seen coming.
- **MAPE** (mean absolute percentage error) — a percentage, reported because papers do, but it balloons whenever the true speed is small, so it should be read next to MAE.
- **Masked** means unknown true speeds are skipped, so an outage never counts as a mistake. Without this, a correct forecast of 67 mph during an outage would be scored as a 67 mph error.
- **Sliced**: by horizon (15/30/60 min), rush hour vs off-peak, by hour of day, and per sensor.
- **As a jam detector**: call "jam" when the predicted speed is below 35 mph and score it with precision (of the jams we called, how many were real), recall (of the real jams, how many we caught) and F1.

Why not accuracy? For the jam detector, about 90% of answers are "free-flowing". A model that **always** says "free" gets 89.7% accuracy while catching zero jams. Accuracy hides failure on rare events; MAE, precision and recall do not.

---

7. Results

Validation weeks (used only to choose when to stop training)

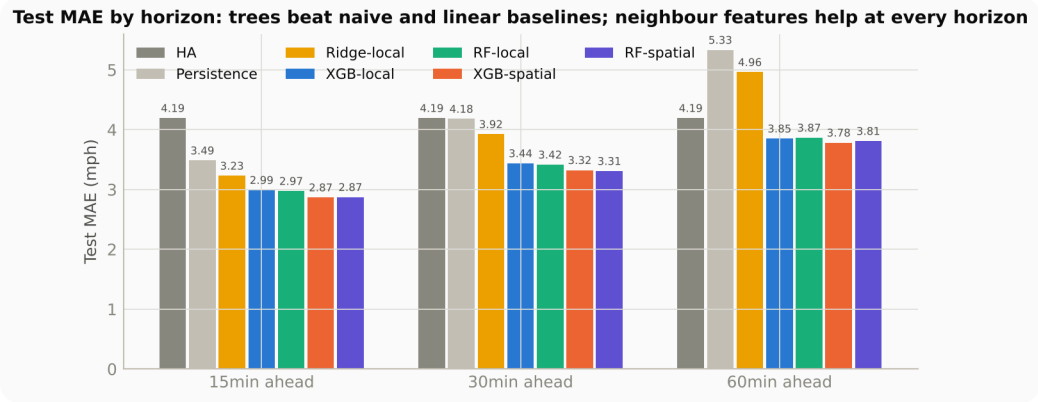
MODEL	MAE 15 MIN	MAE 30 MIN	MAE 60 MIN	RESULT
Historical average	4.32	4.32	4.30	Same at every horizon: ignores the present
Persistence	3.21	3.80	4.75	Great at 15 min, poor at 60: ignores the usual
Ridge (local)	3.10	3.82	4.87	A straight line is not enough
XGBoost (local)	2.85	3.31	3.75	Combines "now" and "usual"
XGBoost (spatial)	**2.70**	**3.16**	**3.63**	**Best → used on test**
Random Forest (local)	2.86	3.32	3.79	Same story as XGBoost
Random Forest (spatial)	2.73	3.16	3.69	A tie with XGBoost

XGBoost with neighbour features was the best model on validation, and the Random Forest with neighbour features was 0.03–0.06 mph behind. Both beat the linear model by a wide margin at 60 minutes, which tells us the "if this *and* that" interactions matter. The ranking is the same on the test set below, which is the evidence that nothing was tuned on the test data.

Test weeks (final check, used once) — MAE in mph

MODEL	15 MIN	30 MIN	60 MIN	RMSE 60 MIN	MAPE 60 MIN
Historical average	4.19	4.19	4.19	7.86	13.1%
Persistence	3.49	4.18	5.33	10.27	14.4%
Ridge (local)	3.23	3.92	4.96	8.44	15.1%
XGBoost (local)	2.99	3.44	3.85	7.26	11.6%
XGBoost (spatial)	**2.87**	**3.32**	**3.78**	**7.07**	**11.2%**
Random Forest (local)	2.97	3.42	3.87	7.25	11.7%
Random Forest (spatial)	2.87	3.31	3.81	7.06	11.3%
DCRNN (Li et al., 2018), published deep graph model on the same data	2.77	3.15	3.60	7.60	10.5%

Three things stand out. **The two yardsticks bracket the problem:** persistence is excellent at 15 minutes but collapses by 60; the historical average is the same at every horizon. **The learned models combine both ideas and win:** at 60 minutes XGBoost (local) is 8% better than the historical average, 28% better than persistence and 22% better than Ridge. **XGBoost and the Random Forest are a tie** (0.01 mph apart; the forest has the slightly lower RMSE), so the choice between them is about training time — XGBoost is three times faster — not accuracy.



Test MAE for every model and horizon.

What the neighbour features add (the key experiment)

Same learner, same rows, same settings — the only change is the eight neighbour columns:

COMPARISON	15 MIN	30 MIN	60 MIN
XGBoost: MAE reduction from adding neighbour features	**-4.1%**	**-3.3%**	**-1.8%**
Random Forest: the same comparison	-3.4%	-3.1%	-1.4%
XGBoost at 60 min, rush hour only			-3.2%
XGBoost at 60 min, off-peak only			-1.1%
Sensors whose 60-min error improved			147 of 207 (up to -7.2%)

The gain is there at every horizon and for both learners, so it is a property of the information, not of one algorithm. It is **three times larger in rush hour** than off-peak — the signature of congestion spreading between sensors. And it is **largest at short horizons and smallest at 60 minutes**, which is exactly what the physics predicts for one-hop features: a jam tail moving at 15–20 km/h crosses the 1–3 km to the next sensor in 5–15 minutes, so the neighbours' current state says most about the next 15–30 minutes, while what determines a sensor an hour ahead is still several hops away.

Rush hour vs off-peak — MAE at 60 minutes, mph

MODEL	RUSH HOUR	OFF-PEAK	RUSH ÷ OFF-PEAK
Historical average	6.53	3.60	1.8×
Persistence	7.79	4.71	1.7×
Ridge (local)	7.64	4.28	1.8×
XGBoost (local)	5.86	3.34	1.8×
XGBoost (spatial)	**5.67**	**3.31**	**1.7×**
Random Forest (spatial)	5.73	3.33	1.7×

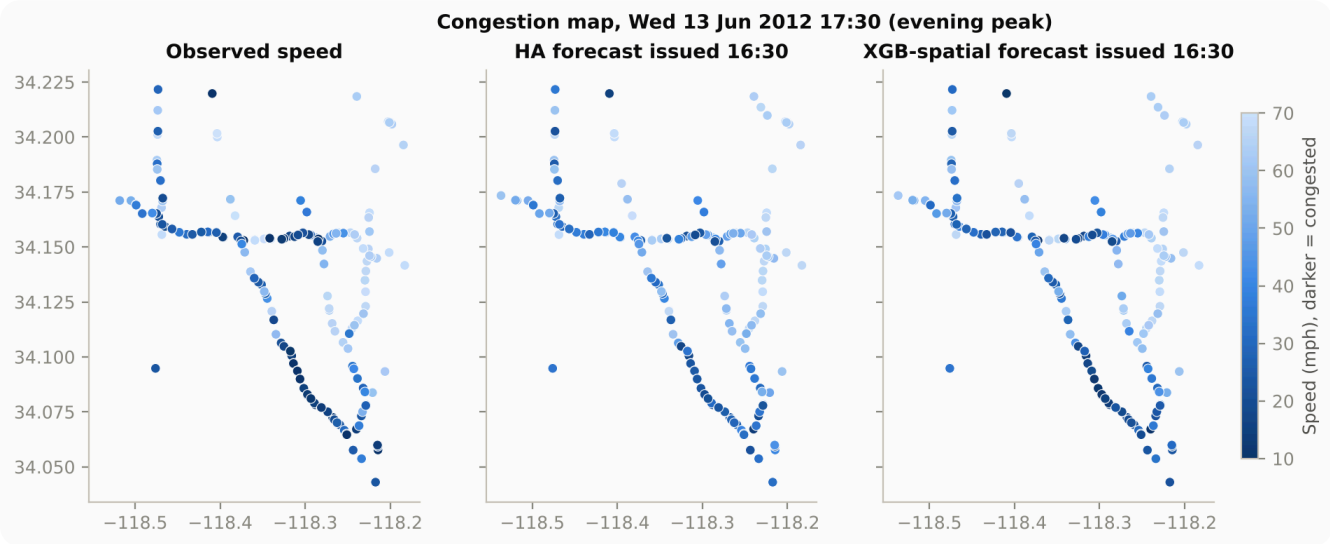
Every model's error roughly doubles in the peaks (weekdays 07–10 and 16–19). Almost all of the extra error is in jammed traffic below 35 mph; above it, even persistence is close to the best

model. Rush hour is where the forecast matters most and where there is most left to gain.

The forecast as a jam detector (60 minutes ahead, jam = below 35 mph)

	REAL JAM	REAL FREE-FLOWING
Model said jam	78,082 (caught)	13,604 (false alarms)
Model said free	50,475 (missed)	1,100,147 (correct)

Precision **0.85** (when it says jam, it is right 85% of the time), recall **0.61** (it catches 61% of the jams an hour ahead), F1 **0.71** — against F1 0.64 for both the historical average and persistence. Model accuracy is 94.8%; always saying "free" would score 89.7%, which is why accuracy is not the headline. **Recall is the hard part:** a jam that has not yet reached a sensor is invisible in that sensor's own history, which is exactly the information a graph model would carry over.



Wednesday 13 June 2012 at 17:30: what was observed (left), what the historical average expected (middle), and what XGBoost (spatial) forecast an hour earlier (right). Dark = jammed.

What the model actually uses

INPUT	SHARE OF IMPORTANCE	WHAT IT IS
`lag_0`	23%	The speed right now
`down_mean_lag0`	**8%**	**Speed of the downstream neighbours (the sensors ahead) right now**
`ha_now`	7%	The usual speed at this time of week
`win_mean`	6%	Average of the last hour
`up_mean_lag0`	6%	Speed of the upstream neighbours right now
`nbr_frac_congested`	5%	Share of neighbours that are jammed
`hour`, `is_weekend`, `tod_cos`, `tod_sin`	14%	The clock
All eight neighbour features together	26%	The model does use the neighbourhood, and the sensors *ahead* most — as the physics predicts

Importance = share of XGBoost's split gain, averaged over all 207 sensors and 3 horizons.

---

8. Key insights

1. Neighbours help — most in rush hour

Adding eight neighbour numbers to the same XGBoost lowers the error at every horizon, for 147 of 207 sensors, and three times more in the peaks than off-peak. The model's second most-used input is the downstream neighbours. Congestion really does spread between sensors, and a model that looks only at one sensor misses it.

2. The help fades at 60 minutes — and that is the gap for Phase 2

The published graph models beat single-sensor models by margins that *grow* with the horizon; our hand-built one-hop features help *least* at 60 minutes. The two facts agree: what matters an hour ahead is several hops away, along paths whose importance changes with the time of day — something a multi-layer, attention-weighted network can represent and a fixed one-hop average cannot. Our measurement is a lower bound on the value of network information, and the horizon at which it fades is where a Graph Attention Network should add the most.

3. Rush hour is the hard part

Error doubles in the peaks and sits almost entirely below 35 mph. The jam detector catches 61% of jams an hour ahead: the ones it misses have not yet reached the sensor's own history. Free-flowing traffic is easy — "the speed stays the same" is already nearly the best forecast there.

4. XGBoost and the Random Forest are a tie

0.01 mph apart on the test set; the forest has the slightly lower RMSE; XGBoost trains three times faster. We did not run a significance test, so we do not claim one is better.

5. The numbers can be trusted

Split by date with a gap; settings fixed in advance; the validation weeks rank the models in exactly the same order as the test weeks; early stopping kept about 130 of 600 trees. One command reruns everything from the raw file to every figure and every results table this report quotes, and eight automated tests check the split, the features and the scoring.

6. Accuracy would have hidden all of this

For the jam detector, "always free" scores 89.7% accuracy. We report MAE in mph, precision and recall instead.

---

9. Problems we found and fixed in our own pipeline

While checking our work, we found four problems and fixed them before producing the final numbers:

1. **Our first write-up claimed the wrong direction.** We expected the neighbour gain to grow with the horizon, as it does for graph networks, and drafted the text that way. The numbers said the opposite (-4.1% at 15 min, -1.8% at 60). We rewrote the claim to match the results and worked out why one-hop features behave this way (Section 5).
2. **The road-graph test was testing nothing.** Our first unit test used a toy graph in which all distances were equal, so every link weight fell below the cut-off and the adjacency matrix was empty — the test passed for the wrong reason. We rebuilt it with realistic multi-hop distances; it now checks the real behaviour, and the rebuilt matrix matches the published one exactly.
3. **The data file would not open with the usual tool.** The released file is an old-format HDF5 that current pandas cannot read directly. We read it with the lower-level library and documented the file's checksum so anyone can verify they have the same data.
4. **We considered filling the outages and decided against it.** Forward-filling the 0 mph gaps looked harmless until we measured the runs: the longest lasts almost six days, so filling would carry a stale reading through many rush hours. We kept the gaps as "unknown" and let the tree models handle them.

---

10. Limitations

- **One city, four months of 2012.** We claim nothing about other seasons or cities, and the test weeks include the start of the summer holidays.
- **Settings fixed in advance.** Model settings were not tuned per sensor; a search on the validation weeks might narrow the XGBoost / Random Forest difference.
- **Unknown readings are skipped, not filled.** A live system would need a rule for sensors that go dark.
- **MAPE is dominated by low speeds** and should be read alongside MAE.
- **The neighbour features depend on the published road map.** Errors in the distance table would propagate.

- **No significance test.** The two best models are 0.01 mph apart.
- **Speed only.** METR-LA has no flow or occupancy; the PeMS source (Chen et al., 2001) does, and is the natural extension.
- **No control experiment yet.** Any claim about traffic-signal control waits for a simulator in Phase 2.

---

11. Next steps (Phase 2)

1. **Train a Graph Attention Network** on the same road graph, with the same one-hour history, horizons, split and masked scoring, so the comparison with Phase 1 is exact.
2. **Check the attention against the physics:** does the network learn to listen to downstream neighbours in rush hour, as the backward wave predicts? Explore an LWR-based penalty as the hybrid ML/physics component.
3. **Ablations** on how many hops the network looks out, which adjacency it uses, and how much history it sees.
4. **Report the simple baselines** (historical average, persistence, XGBoost spatial) next to the deep model, always.
5. **If time permits, a simulated multi-agent signal-control experiment** in SUMO or CityFlow that uses the forecasts — reported strictly as a simulation.

---

12. Team and contributions

MEMBER	ENROLLMENT	WORK
Kartik Madaan	2401010212	Data pipeline: acquisition, outage masking, road-graph construction, repository and reproducibility; exploratory data analysis and figures
Arhan Alam	2401020010	Feature engineering, LWR-motivated neighbour features, chronological split and leakage tests
Chirag Lalwani	2401020093	Model training: per-sensor XGBoost with early stopping, multi-output Random Forest, Ridge, settings
Siddhanth Sadshiv Raikar	2401020110	Models (baselines, training runs, validation evidence) and the report: literature review, assembly and editing
Abhijeet Dey	2401010014	Evaluation and results: masked metrics, rush hour vs off-peak, the neighbour ablation, jam detection and congestion maps

All members reviewed every component and can present and defend the full pipeline.

Code: github.com/<your-account>/traffic-flow-phase1 — one command (`run_all.sh`) reruns data → analysis → models → figures → results tables.

Glossary

TERM	SIMPLE MEANING
Sensor / reading	A loop detector buried in the road; every five minutes it reports the average speed of the cars over it
Horizon	How far ahead we predict: 15, 30 or 60 minutes
Train / validation / test	Data to learn from / data to decide when to stop training / data for the final, one-time check
Leakage	When the model sees information it would not have in real life, making its score look too good
Unknown (NaN)	A reading we do not trust (the sensor was offline); it is skipped, never counted as 0
Masked	Scoring only the cells whose true speed is known
MAE / RMSE / MAPE	Average mistake in mph / the same but punishing big misses more / the mistake as a percentage
Historical average	The usual speed of this sensor at this time of week
Persistence	"The speed stays what it is now"
Lag	A past value: lag 3 = the reading 15 minutes ago
Feature	A number the model uses as input
Local / spatial	Inputs about the sensor itself only / those plus inputs about its neighbours
Upstream / downstream	The sensors behind you (traffic comes from there) / the sensors ahead of you (traffic goes there)
LWR model	The 1955 theory that treats traffic like a fluid; it predicts that jams grow backwards
Early stopping	Stop adding trees when the validation error stops improving
Feature importance	How much the trained model relies on each input
Jam	Speed below 35 mph (the Caltrans definition of congested)
Precision / recall / F1	Of the jams we called, how many were real / of the real jams, how many we caught / one number combining both
Ablation	Removing one thing (here, the neighbour features) to measure what it was worth

References

1. Y. Li, R. Yu, C. Shahabi, Y. Liu. "Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting." *ICLR*, 2018.

2. M. J. Lighthill, G. B. Whitham. "On Kinematic Waves II. A Theory of Traffic Flow on Long Crowded Roads." *Proceedings of the Royal Society of London, Series A*, 229, pp. 317–345, 1955.
3. P. I. Richards. "Shock Waves on the Highway." *Operations Research*, 4(1), pp. 42–51, 1956.
4. B. D. Greenshields. "A Study of Traffic Capacity." *Highway Research Board Proceedings*, 14, pp. 448–477, 1935.
5. M. Treiber, A. Kesting. *Traffic Flow Dynamics: Data, Models and Simulation*. Springer, 2013.
6. E. I. Vlahogianni, M. G. Karlaftis, J. C. Golias. "Short-Term Traffic Forecasting: Where We Are and Where We're Going." *Transportation Research Part C*, 43, pp. 3–19, 2014.
7. Y. Zhang, A. Haghani. "A Gradient Boosting Method to Improve Travel Time Prediction." *Transportation Research Part C*, 58, pp. 308–324, 2015.
8. T. Chen, C. Guestrin. "XGBoost: A Scalable Tree Boosting System." *Proceedings of the 22nd ACM SIGKDD Conference*, pp. 785–794, 2016.
9. L. Breiman. "Random Forests." *Machine Learning*, 45(1), pp. 5–32, 2001.
10. A. E. Hoerl, R. W. Kennard. "Ridge Regression: Biased Estimation for Nonorthogonal Problems." *Technometrics*, 12(1), pp. 55–67, 1970.
11. C. Bergmeir, J. M. Benítez. "On the Use of Cross-Validation for Time Series Predictor Evaluation." *Information Sciences*, 191, pp. 192–213, 2012.
12. P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio. "Graph Attention Networks." *ICLR*, 2018.
13. C. Zhang, J. J. Q. Yu, Y. Liu. "Spatial-Temporal Graph Attention Networks: A Deep Learning Approach for Traffic Forecasting." *IEEE Access*, 7, pp. 166246–166256, 2019.
14. W. Jiang, J. Luo. "Graph Neural Network for Traffic Forecasting: A Survey." *Expert Systems with Applications*, 207, 117921, 2022.
15. C. Chen, K. Petty, A. Skabardonis, P. Varaiya, Z. Jia. "Freeway Performance Measurement System: Mining Loop Detector Data." *Transportation Research Record*, 1748, pp. 96–102, 2001.
16. H. V. Jagadish et al. "Big Data and Its Technical Challenges." *Communications of the ACM*, 57(7), pp. 86–94, 2014.
17. P. A. Lopez et al. "Microscopic Traffic Simulation using SUMO." *IEEE ITSC*, pp. 2575–2582, 2018.
18. H. Zhang et al. "CityFlow: A Multi-Agent Reinforcement Learning Environment for Large Scale City Traffic Scenario." *The Web Conference (WWW)*, pp. 3620–3624, 2019.